

Módulo 5 – Capítulo 04 – Sección 01

Patrones de integración: síncrono, asíncrono, event-driven y por lotes

Añadir capacidades de IA a una aplicación existente no es simplemente agregar llamadas al LLM en el código que ya existe: es una decisión arquitectónica sobre cómo el sistema de IA se integra en el flujo de datos, el modelo de concurrencia y el contrato de latencia que la aplicación ya tiene. El patrón de integración equivocado puede hacer que un sistema de IA correctamente implementado resulte inutilizable en producción: una tarea de análisis de documentos que toma 30 segundos no puede servirse con una conexión HTTP que el cliente espera síncrona, ni un chat conversacional puede implementarse con un job de procesamiento nocturno.

El patrón síncrono —request/response— es el más simple y el más adecuado para interacciones conversacionales con latencia tolerable de 2 a 10 segundos y volumen bajo a medio. El cliente hace un POST, el servidor llama al LLM, y la respuesta llega en la misma conexión HTTP antes de que el timeout del cliente expire. La variante más común en producción combina el patrón síncrono con streaming SSE: `StreamingResponse` en FastAPI transmite los tokens del LLM al cliente a medida que se generan, eliminando la espera en blanco inicial sin cambiar la naturaleza síncrona del request. Este patrón es suficiente para la mayoría de los chatbots, asistentes de redacción y herramientas de análisis de texto con latencia de usuario aceptable.

El patrón asíncrono desacopla la recepción de la solicitud de la entrega del resultado. El cliente envía el request y recibe inmediatamente un `job_id`; la tarea se ejecuta en background; el cliente consulta el estado via polling (`GET /jobs/{job_id}/status`) o recibe notificación cuando la tarea termina via webhook. Este patrón es correcto para tareas que toman entre 30 segundos y varios minutos —análisis de documentos largos, procesamiento de audios, generación de reportes— donde mantener la conexión HTTP abierta durante ese tiempo es costoso e inestable. Celery con broker Redis o RabbitMQ, o el servicio ARQ (Async Redis Queue) para Python asyncio, implementan este patrón con workers que consumen tareas de una cola y actualizan el estado en la base de datos.

El patrón event-driven con brokers de mensajes como Kafka, RabbitMQ o AWS SQS permite escalar horizontalmente el procesamiento de IA distribuyendo el trabajo entre múltiples workers consumidores, con control de backpressure y dead letter queues para manejo robusto de fallos. Este patrón es correcto cuando el pipeline de IA es un componente dentro de un sistema de eventos más amplio: un evento de "documento cargado" dispara el pipeline de análisis; el resultado del análisis genera un evento de "análisis completado" que otros sistemas consumen. La arquitectura event-driven desacopla completamente el productor del procesamiento, permitiendo escalar la capacidad de procesamiento de IA independientemente de la aplicación que genera los eventos.

El batch processing nocturno procesa grandes volúmenes de datos en ventanas de tiempo donde la latencia por item individual no importa. Scripts que leen N registros de una base de datos, los procesan con el LLM en paralelo usando `asyncio.gather()` con un semáforo que controla la concurrencia máxima, y escriben los resultados de vuelta son apropiados para enriquecimiento masivo de datos, clasificación de corpus, y generación de embeddings para indexación. La optimización de throughput en batch incluye usar el Batch API de Anthropic — que procesa requests con hasta 50% de descuento a cambio de un SLA de latencia de 24 horas — para tareas que no requieren resultados en tiempo real.

Aspectos técnicos de los patrones de integración

- **Síncrono con streaming:** FastAPI `StreamingResponse` con `media_type="text/event-stream"` y un generador async que re-transmite los chunks del proveedor al cliente; configurar `timeout=httpx.Timeout(read=120)` para respuestas largas.
- **Asíncrono con Celery:** task `@app.task` que llama al LLM, actualiza el estado en Redis o PostgreSQL, y opcionalmente dispara un webhook al callback URL del cliente; polling con `GET /tasks/{task_id}` o eventos via WebSocket.
- **Event-driven con SQS + Lambda:** mensaje en SQS dispara Lambda que procesa con el LLM y escribe el resultado en DynamoDB o SNS; el dead letter queue captura los mensajes que fallan tras `maxReceiveCount` intentos.
- **Batch con control de concurrencia:** `asyncio.Semaphore(MAX_CONCURRENT=20)` limita las llamadas paralelas al LLM para respetar el rate limit del proveedor; `asyncio.gather(*tasks, return_exceptions=True)` procesa todos los items en paralelo respetando ese límite.
- **Elección del patrón:** latencia < 5s y UX conversacional → síncrono con streaming; análisis de documento individual → asíncrono con webhook; enriquecimiento de millones

de registros → batch; pipeline de eventos en tiempo real → event-driven.

Nota del Arquitecto: *El error de diseño más frecuente en proyectos de IA nuevos es usar el patrón síncrono para tareas que son inherentemente asíncronas —análisis de documentos de 50 páginas, generación de reportes, transcripción de audio— porque "es más simple al inicio". El resultado es un timeout en producción que fuerza una reescritura bajo presión. El patrón de integración determina el diseño del sistema completo; cambiarlo después de estar en producción es una refactorización mayor. Vale la pena elegirlo correctamente desde el primer sprint.*

El patrón de integración elegido determina no solo la arquitectura de la capa de IA sino la del sistema completo que la rodea: el modelo de autenticación, el manejo de errores, la estrategia de escalabilidad y la experiencia del usuario. La siguiente sección aborda cómo exponer las capacidades de IA hacia otras aplicaciones y servicios mediante APIs REST diseñadas para durar.

Principio rector: El patrón de integración es la decisión arquitectónica más importante al añadir IA a una aplicación; elegirlo correctamente desde el inicio —síncrono, asíncrono, event-driven o batch según la latencia, volumen y acoplamiento requeridos— determina la viabilidad del sistema en producción.

Módulo 5 – Capítulo 04 – Sección 02

APIs REST para exponer capacidades de IA a otras aplicaciones

La forma más efectiva de hacer que las capacidades de IA estén disponibles para otros sistemas —frontends web, aplicaciones móviles, microservicios internos, herramientas de terceros— es encapsularlas detrás de una API REST con contratos bien definidos. Este encapsulamiento oculta los detalles de implementación: qué proveedor de LLM usa el sistema, qué versión de modelo, qué estructura de prompt, qué framework de orquestación. Los sistemas consumidores solo conocen el contrato de la API —el schema de entrada y salida, los códigos de error, las garantías de versionado— y pueden adaptarse a cambios de implementación interna sin modificación.

FastAPI es el framework Python dominante para este propósito por tres razones concretas. Los modelos Pydantic se traducen automáticamente en schemas de validación de request, documentación OpenAPI/Swagger generada en `/docs`, y serialización tipada de responses, sin ningún código adicional. El soporte nativo de `async/await` hace que los endpoints puedan llamar a las APIs de LLM de forma no bloqueante, manejando múltiples requests concurrentes sin hilos adicionales. Y `StreamingResponse` permite transmitir tokens al cliente con tres líneas de código, lo que en otros frameworks requiere configuración significativa.

El diseño del contrato de la API de IA tiene consideraciones específicas que no aplican a APIs convencionales. El schema de request debe incluir los campos de contenido (el texto o pregunta del usuario, los documentos a procesar, el historial conversacional) más campos de configuración opcionales que dan flexibilidad sin exponer complejidad interna: `tone` para aplicaciones de redacción, `format` para aplicaciones de análisis, `language` para sistemas multilingües. El schema de response debe incluir siempre los datos de resultado más un campo `metadata` con información de operación: `model_used`, `input_tokens`, `output_tokens`, `latency_ms`, `request_id`. Esta metadata es invaluable para debugging, facturación interna y análisis de comportamiento, y añadir estos campos al contrato desde el inicio es mucho más simple que hacerlo retroactivamente cuando múltiples sistemas ya consumen la API.

La autenticación de la API interna de IA debe implementarse con JWT o API keys propias — no las del proveedor de LLM, que son credenciales de infraestructura— con scopes que definen qué capacidades puede usar cada cliente interno. Un servicio de frontend puede tener scope `chat:send` pero no `analysis:batch`; un servicio de procesamiento interno puede tener `analysis:batch` sin necesidad de `chat:send`. Esta granularidad de permisos aplica el principio de mínimo privilegio al nivel de la API de IA, independientemente de los permisos que cada servicio tenga hacia el proveedor externo.

El versionado de endpoints con prefijo `/v1/` desde el inicio establece la política explícita de que el contrato de la versión activa se mantiene estable durante al menos doce meses tras publicar la siguiente. Esta garantía permite que los sistemas consumidores migren a nuevas versiones sin urgencia, y que el equipo de IA evolucione la implementación interna — cambiando el modelo, el prompt, el framework— sin romper integraciones existentes.

Componentes principales de una API REST de IA

- **Schema de request con Pydantic:** modelos de entrada con validación de campos obligatorios (`user_message: str = Field(min_length=1, max_length=4096)`), tipos enumerados para parámetros opcionales, y documentación de campo que se propaga a la documentación OpenAPI.
- **Middleware de autenticación:** `Depends(verify_jwt_token)` en FastAPI que valida el token antes de ejecutar el endpoint, extrae los scopes del payload JWT, y verifica que el scope requerido esté presente; rate limiting por API key usando Redis.
- **Schema de response estandarizado:** estructura `{"data": {...}, "metadata": {"model": "...", "input_tokens": N, "output_tokens": N, "latency_ms": N, "request_id": "..."}, "error": null}` consistente para éxito y fallo, con el mismo envelope en ambos casos.
- **Versionado semántico de endpoints:** prefijo `/v1/` en todos los endpoints con política explícita de backward compatibility; nuevas capacidades se añaden en `/v2/` mientras `/v1/` permanece operativa durante el período de migración.
- **Health checks:** endpoint `/health` que verifica la conectividad con el proveedor de LLM haciendo una llamada de prueba y devuelve la latencia; `/readiness` que incluye además la conectividad con bases de datos y caches; usados por Kubernetes para routing de tráfico.

La API REST de IA es el contrato entre las capacidades de IA y el resto del sistema; su estabilidad, documentación y versionado determina la autonomía con que el equipo de IA

puede evolucionar la implementación interna sin coordinar con todos los equipos consumidores en cada cambio. La siguiente sección aborda la integración de IA con las fuentes de datos que el sistema ya tiene: bases de datos relacionales, documentales, y vectoriales.

Para recordar: La API REST de IA es el contrato con el resto del sistema; estabilizarla con versionado explícito, schema de metadata estandarizado y documentación OpenAPI actualizada protege a los equipos consumidores de los cambios internos de implementación del modelo o el prompt.

Módulo 5 – Capítulo 04 – Sección 03

Integración con bases de datos relacionales y documentales

Las aplicaciones de IA empresariales raramente operan sobre texto puro: necesitan datos del usuario almacenados en PostgreSQL, historial de transacciones en MySQL, documentos en MongoDB, o búsqueda full-text en Elasticsearch. La integración de estas fuentes de datos con el LLM es el puente entre el modelo y el contexto de negocio que hace que la respuesta sea relevante para ese usuario específico, en ese momento específico, con ese historial de interacciones. Un sistema de atención al cliente que no tiene acceso al historial de pedidos del usuario antes de construir el prompt no puede personalizar la respuesta; un asistente legal que no recupera los documentos relevantes del expediente solo puede ofrecer respuestas genéricas.

El patrón más común en aplicaciones conversacionales es la recuperación de datos del usuario antes de construir el prompt: una consulta SQL que obtiene el perfil, el historial de interacciones, el estado de la cuenta o los productos adquiridos, y lo serializa en texto o JSON estructurado que se inserta en el contexto del LLM. `SELECT u.name, u.plan, o.status, o.items FROM users u JOIN orders o ON u.id = o.user_id WHERE u.id = $1 ORDER BY o.created_at DESC LIMIT 5` seguido de un formateador que convierte los registros en texto enriquece el prompt con el contexto real del usuario en segundos. La clave es que esta consulta es parte de la preparación del prompt, no del manejo de la respuesta, y su latencia se suma a la latencia total del request de IA.

La integración con bases de datos vectoriales —pgvector, Qdrant, Chroma— implementa la capa de recuperación semántica del patrón RAG. pgvector merece atención especial: al ser una extensión de PostgreSQL, no requiere infraestructura adicional cuando el proyecto ya usa Postgres, y su integración con SQLAlchemy es directa. `CREATE EXTENSION IF NOT EXISTS vector; ALTER TABLE documents ADD COLUMN embedding vector(1536)` añade capacidad vectorial a una tabla existente. La búsqueda por similitud coseno usa el operador nativo: `ORDER BY embedding <=> $1::vector LIMIT 10`. Para proyectos que comienzan con RAG y no quieren gestionar un servicio de base de datos vectorial adicional, pgvector es el punto de

entrada natural antes de migrar a Qdrant o Pinecone cuando el volumen o los requisitos de rendimiento lo justifiquen.

La escritura de resultados del LLM de vuelta en la base de datos debe tratarse con las mismas garantías de consistencia que cualquier escritura transaccional. Si el pipeline de IA clasifica un ticket de soporte y actualiza su categoría en la base de datos, esa actualización debe ocurrir en la misma transacción que cualquier otro cambio de estado del ticket: o se guarda el resultado completo del pipeline o no se guarda nada, evitando estados parciales difíciles de diagnosticar. El JSON que devuelve el LLM debe validarse con el schema Pydantic esperado antes de intentar insertarlo en la base de datos, rechazando y loggeando los casos donde el LLM devolvió un formato inesperado en lugar de insertar datos malformados silenciosamente.

La caché de resultados con Redis es la optimización más efectiva para queries frecuentes cuya respuesta del LLM es determinista para las mismas entidades: si la clasificación de la categoría de un producto no cambia entre requests del mismo producto, cachear el resultado con TTL alineado a la frecuencia de cambio del catálogo elimina llamadas redundantes al LLM. `redis.setex(f"classify:{product_id}", 3600, result)` con un TTL de una hora es apropiado para datos que cambian diariamente; `redis.setex(f"classify:{product_id}", 86400, result)` para datos que cambian semanalmente.

Aspectos técnicos de la integración con bases de datos

- **pgvector con SQLAlchemy:** columna `Vector(1536)` en el modelo SQLAlchemy, operador `<=>` para distancia coseno en queries de búsqueda, índice `CREATE INDEX ON documents USING ivfflat (embedding vector_cosine_ops)` para búsqueda aproximada eficiente a escala.
- **SQLAlchemy async:** `AsyncSession` con `asyncpg` para no bloquear el event loop y asyncio durante consultas a la base de datos; `async with async_session() as session: result = await session.execute(select(User).where(User.id == user_id))` es el patrón estándar.
- **MongoDB para datos semiestructurados:** colección de logs de prompts y respuestas con schema `{request_id, model, input_tokens, output_tokens, latency_ms, prompt_text, response_text, created_at, user_id, feature}`; indexar `user_id` y `created_at` para queries frecuentes de historial.
- **Connection pooling:** `pool_size=10`, `max_overflow=20` en SQLAlchemy para no agotar las conexiones de la base de datos bajo carga concurrente; el pool debe configurarse antes

del primer request de producción, no después de la primera crisis de conexiones agotadas.

- **Caché de resultados con Redis:** `redis.get(cache_key)` antes de la llamada al LLM; si existe el valor, devolverlo sin llamar al API; si no existe, llamar al LLM, almacenar el resultado con TTL apropiado y devolverlo; loggear cache hit/miss como métrica operativa.

Separar la capa de acceso a datos de la lógica de llamada al LLM con interfaces claras — repositorios o servicios de datos que exponen métodos semánticos como `get_user_context(user_id)` en lugar de queries SQL en el código del pipeline— permite testear cada capa independientemente con mocks y evita el acoplamiento entre el schema de la base de datos y los prompts.

Buena práctica: La separación entre la capa de recuperación de datos y la capa de llamada al LLM con interfaces semánticas es la práctica más impactante para la testeabilidad y el mantenimiento a largo plazo; los cambios en el schema de la base de datos o en la estructura del prompt no deben propagarse al código de la capa contraria.

Módulo 5 – Capítulo 04 – Sección 04

Manejo de estado y sesión en aplicaciones conversacionales

Una de las tensiones fundamentales en el diseño de aplicaciones conversacionales con LLMs es la asimetría entre el modelo de comunicación preferido de las APIs REST —stateless, cada request independiente— y la naturaleza inherentemente stateful de una conversación: cada turno requiere acceso al historial previo para mantener coherencia, referencias a turnos anteriores, y continuidad del contexto establecido. Resolver esta tensión correctamente desde el diseño inicial es crítico; hacerlo mal es una de las fuentes más frecuentes de refactorizaciones dolorosas cuando el sistema escala más allá de un único usuario simultáneo.

La estrategia de almacenamiento del historial conversacional tiene tres opciones principales con trade-offs diferentes. Almacenamiento en el cliente —el frontend envía todo el historial en cada request— elimina la necesidad de persistencia en el servidor y simplifica la arquitectura, pero transfiere al cliente la responsabilidad de mantener un historial potencialmente grande y expone el contenido completo de la conversación en cada request. Almacenamiento en memoria del servidor es la solución más simple en desarrollo —un diccionario Python en el proceso del servidor— pero es incompatible con múltiples instancias del servicio y pierde el estado al reiniciar. Almacenamiento externo en Redis (para sesiones con TTL de horas o días) o en base de datos relacional/documental (para historial permanente) es el patrón correcto para producción: escala horizontalmente, sobrevive reinicios del servicio, y permite que el historial sea accesible desde cualquier instancia del backend.

La ventana de contexto del modelo impone un límite físico sobre el historial que puede incluirse en cada llamada. Con Claude 3.5 Sonnet y 200.000 tokens de ventana, una sesión de chat intensa con turnos de 500 tokens cada uno puede acumular el historial de 400 turnos sin truncar —suficiente para conversaciones de muchas horas. Pero el costo de incluir ese historial completo en cada request es real: 200.000 tokens de entrada a \$3/M cuestan \$0.60 por llamada, lo que hace inviable económicamente incluir el historial completo en sistemas de alto volumen. Las estrategias de compresión del historial no son opcionales en producción: son decisiones de diseño que determinan el equilibrio entre coherencia de la conversación y costo por turno.

La compresión por sliding window —mantener los últimos K turnos completos y descartar los anteriores— es la implementación más simple y la más adecuada cuando la conversación es lineal y los turnos recientes son suficientes para el contexto. `messages[-2*K:]` selecciona los K últimos intercambios de usuario-asistente. La compresión por summarization usa el propio LLM para generar un resumen del historial cuando supera un umbral: `if count_tokens(messages) > MAX_CONTEXT_TOKENS: summary = llm.summarize(messages[:-K]); messages = [summary_message] + messages[-K:]`. El costo del resumen —una llamada adicional al LLM— se amortiza en múltiples turnos futuros que usan el historial comprimido. La caché RAG sobre historial recupera solo los turnos semánticamente relevantes para la pregunta actual usando embeddings, ideal para sesiones muy largas donde el usuario puede referenciar temas discutidos muchos turnos antes.

Componentes principales del manejo de estado conversacional

- **Estructura de sesión:** `{session_id, user_id, created_at, last_active, messages: [{role, content, timestamp, input_tokens, output_tokens}], total_tokens, metadata: {model, prompt_version}}` almacenada como JSON en Redis o MongoDB.
- **Redis con TTL:** `redis.setex(f"session:{session_id}", 86400, json.dumps(session_data))` para sesiones con 24 horas de TTL; `redis.expire()` renueva el TTL en cada interacción activa; las sesiones inactivas expiran automáticamente sin job de limpieza.
- **Compresión por sliding window:** `messages = messages[-2*K:]` donde K es el número de pares usuario-asistente a mantener; es el primer mecanismo de control de costo a implementar antes de técnicas más complejas.
- **Compresión por summarization:** umbral configurable `MAX_CONTEXT_TOKENS`; cuando el historial supera el umbral, una llamada al LLM con `temperature=0` genera el resumen; el resumen reemplaza los turnos comprimidos como mensaje de rol `system` o `assistant` al inicio del historial.
- **Sesiones multi-dispositivo:** usar `user_id` como clave de sesión en lugar de `session_id` de browser cuando la consistencia entre dispositivos es un requisito; locking optimista con Redis para prevenir escrituras concurrentes cuando el usuario accede desde múltiples dispositivos simultáneamente.

Nota del Arquitecto: El diseño de la estrategia de sesión antes del primer sprint de desarrollo no es una optimización prematura: es una decisión que afecta el costo por

usuario, la coherencia de la conversación y la escalabilidad del servicio. Cambiar de historial en memoria a historial en Redis después de estar en producción requiere migrar sesiones activas, actualizar el código de todos los endpoints que leen el historial, y gestionar el período de transición —exactamente el tipo de refactorización bajo presión que puede evitarse con quince minutos de diseño inicial.

La gestión del historial conversacional determina directamente el costo de cada turno y la calidad de la coherencia del diálogo. La siguiente sección aborda los feature flags, la herramienta más poderosa para desplegar nuevas capacidades de IA gradualmente y revertirlas instantáneamente cuando se detectan problemas.

Principio rector: La estrategia de gestión de sesión determina directamente el costo por turno conversacional y la calidad de la coherencia del diálogo; diseñarla antes de la primera línea de código del frontend previene refactorizaciones costosas cuando el sistema escala más allá del prototipo inicial.

Módulo 5 – Capítulo 04 – Sección 05

Feature flags y despliegue gradual de capacidades de IA

Los feature flags —también llamados feature toggles— son la herramienta de control de riesgo más rápida disponible para sistemas de IA en producción: permiten activar o desactivar capacidades, cambiar parámetros de configuración o redirigir tráfico entre versiones de modelo en segundos, sin redeploy de código, sin ventana de mantenimiento, y con reversión instantánea ante cualquier señal de degradación. En el ciclo de vida de un sistema de IA, esta capacidad es especialmente valiosa porque los problemas de calidad son frecuentemente sutiles —el sistema continúa respondiendo sin errores de runtime mientras la calidad se degrada— y la velocidad de detección y rollback determina cuántos usuarios son expuestos al problema.

Los sistemas de feature flags modernos —LaunchDarkly, Unleash (open source), Flagsmith, AWS AppConfig— almacenan la configuración de los flags en un backend centralizado y la distribuyen a los servicios vía SDK con latencia de actualización de milisegundos a segundos, sin requerir redeploy. La evaluación de un flag es una llamada local al SDK que consulta el estado cacheado: `client.is_enabled("new-llm-model", context={"userId": user_id, "properties": {"plan": "enterprise"}})`. El contexto de evaluación permite targeting sofisticado: activar el flag solo para usuarios del plan enterprise, solo para un porcentaje determinístico de users (hasheado por `user_id` para que el mismo usuario reciba siempre el mismo tratamiento), o solo para una región geográfica específica.

En el contexto específico de IA, los feature flags tienen aplicaciones que no existen en sistemas de software tradicionales. El modelo de LLM activo puede ser un valor de configuración leído del sistema de flags en lugar de una constante en el código: `model_name = config_client.get_string("active_llm_model", "claude-3-5-sonnet-20241022")`. Cambiar el modelo para el 10% del tráfico para hacer A/B testing requiere actualizar el flag en la consola web, sin tocar código ni hacer un deploy. Desactivar un agente autónomo que está tomando decisiones erróneas —por ejemplo, un agente de booking que está creando reservas duplicadas— puede hacerse en menos de un minuto desde que se detecta el problema, reduciendo el número de usuarios afectados mientras se diagnostica y corrige la causa raíz.

El rollout gradual porcentual —exponer el nuevo sistema al 5% de usuarios, monitorear métricas de calidad durante 24 horas, escalar al 25%, luego al 50%, luego al 100%— es el patrón más seguro para lanzar cambios significativos en sistemas de IA. A diferencia del blue-green deployment que cambia del 0% al 100% en un momento, el rollout porcentual permite detectar problemas con un impacto acotado y revertir antes de escalar. La condición para que este patrón funcione es tener las métricas de calidad instrumentadas y visibles en tiempo real: si la tasa de feedback negativo del grupo de usuarios del 5% sube del 3% al 15%, el rollout se revierte antes de exponer al 95% restante.

Aspectos técnicos del despliegue gradual con feature flags

- **Rollout porcentual determinístico:** hashear `user_id` y comparar con el umbral de porcentaje garantiza que el mismo usuario recibe siempre el mismo tratamiento (no cambia entre requests); esto es crítico para reproducibilidad de experimentos y coherencia de UX.
- **Flags de configuración de modelo:** `config.get_string("llm_model", default="claude-3-5-sonnet-20241022")` leído del sistema de flags en lugar de hardcodeado; permite cambiar el modelo sin deploy, con audit trail automático de quién hizo el cambio y cuándo.
- **Circuit breaker via feature flag:** integrar el sistema de flags con el sistema de alertas para desactivar automáticamente el flag cuando una métrica de calidad supera un umbral; la respuesta automatizada reduce el tiempo de exposición al problema de minutos a segundos.
- **Testing de ambas ramas:** implementar tests unitarios que ejerciten el código con el flag habilitado y con el flag deshabilitado; el código del flag deshabilitado tiende a deteriorarse silenciosamente si no se testea, comprometiendo la capacidad de rollback cuando se necesita.
- **Limpieza de flags obsoletos:** eliminar del código los flags que han alcanzado el 100% de rollout y han sido estables durante semanas; los flags obsoletos son deuda técnica que aumenta la complejidad del código sin aportar capacidad de control.

Los feature flags son la herramienta de resiliencia más rápida para sistemas de IA en producción: un rollback via flag tarda segundos mientras que un rollback via redeploy puede tardar minutos, y en una degradación activa cada segundo cuenta. La sección siguiente cierra el capítulo con los principios de integración incremental de IA en sistemas existentes —cómo añadir capacidades de IA sin reescribir lo que ya funciona.

Para recordar: Los feature flags son la herramienta de resiliencia más rápida para sistemas de IA en producción: permiten revertir un cambio de modelo, desactivar un agente defectuoso o ajustar parámetros de configuración en segundos sin redeploy, reduciendo el tiempo de exposición a cualquier problema a su mínimo posible.

Módulo 5 – Capítulo 04 – Sección 06

Cierre: integración de IA sin reescribir el sistema existente

La narrativa más peligrosa en la adopción de IA en organizaciones con sistemas existentes es la de "hay que reescribir todo para poder usar IA de forma correcta". Esta narrativa conduce a proyectos de reescritura de seis a doce meses que consumen presupuesto, congelan funcionalidades durante el desarrollo, y entregan el mismo valor que podría haberse entregado en seis semanas con integración incremental. La realidad operacional de cientos de proyectos de IA en producción es que las capacidades más valiosas se construyeron como capas sobre sistemas existentes, sin tocar el código heredado, usando los datos y la infraestructura que ya existían.

El principio guía es el strangler fig aplicado a IA: identificar las funciones de negocio que más se beneficiarían de capacidades de IA —redacción, clasificación, resumen, análisis, búsqueda semántica—, crear endpoints nuevos paralelos a los existentes que implementan esas funciones con LLMs, y migrar el tráfico gradualmente hacia los nuevos endpoints mientras los antiguos permanecen operativos como fallback. Un sistema CRM existente no necesita ser reescrito para añadir un asistente de redacción de emails de seguimiento: basta con un nuevo endpoint `POST /crm/emails/generate` que lee el historial del contacto de la base de datos existente, llama al LLM con ese contexto, y devuelve el borrador al frontend, que añade un botón "Generar con IA" que consume ese endpoint.

El adapter pattern para el LLM es la práctica de diseño más importante para la integrabilidad y la evolucionabilidad. Toda la interacción con el proveedor de LLM se encapsula en una única clase o módulo con una interfaz estable y semántica — `AIAssistant.generate_email_draft(contact, context)`, `DocumentAnalyzer.extract_key_points(document)` — que abstrae el proveedor, el modelo, el prompt y el framework de orquestación detrás de una interfaz de dominio. El código de negocio llama al servicio de IA a través de esta interfaz sin saber si detrás hay Claude o GPT-4, si el prompt cambió en la última iteración, o si el sistema ahora usa RAG en lugar de llamada directa. Cambiar el proveedor, actualizar el modelo o refactorizar el pipeline de IA requiere modificar solo el adaptador.

Los antipatrones que deben evitarse activamente en la integración son: integrar el SDK del LLM directamente en el ORM o en los modelos de dominio existentes, creando acoplamiento entre la lógica de negocio y los detalles del proveedor de IA; usar el objeto de sesión del framework web para almacenar el historial conversacional, que no escala más allá de un único proceso; y hardcodear el proveedor o el modelo en múltiples puntos del código, haciendo que cada migración requiera búsqueda y reemplazo en toda la base de código.

La validación del valor antes del compromiso de recursos es el argumento más poderoso para la integración incremental: un endpoint nuevo que añade IA a una función existente puede estar en producción para un subconjunto de usuarios en dos semanas, produciendo datos reales sobre si el caso de uso tiene el impacto esperado antes de invertir en integración profunda. El monitoreo diferencial —comparar métricas de negocio entre usuarios que usan las nuevas capacidades de IA y los que usan el flujo tradicional— provee la evidencia objetiva para decidir si escalar la inversión o pivotar hacia un caso de uso diferente.

Buenas prácticas de integración incremental

- **Strangler fig para IA:** endpoints nuevos paralelos que implementan funciones de negocio con IA, sin modificar el código heredado; migración gradual del tráfico hacia los nuevos endpoints con feature flags; los endpoints antiguos permanecen como fallback hasta que la nueva versión alcanza el 100% de rollout.
- **Adapter pattern para el LLM:** una única clase o módulo que encapsula toda la interacción con el proveedor; la lógica de negocio llama al servicio de dominio, no al SDK del proveedor; cambiar el proveedor o el modelo requiere modificar solo el adaptador.
- **Separación de concerns:** la lógica de negocio no sabe que existe un LLM; llama a un servicio de "análisis" o "generación" que internamente usa el LLM, manteniendo la abstracción limpia, testeable con mocks, y reemplazable sin tocar el código de negocio.
- **Migraciones de schema sin bloqueo:** añadir columnas nullable para almacenar outputs del LLM en tablas existentes con `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` no bloquea la tabla en PostgreSQL; evitar `ALTER TABLE` que reescribe la tabla completa en sistemas con carga de producción.
- **Monitoreo diferencial:** comparar métricas de negocio entre el grupo de usuarios con IA habilitada y el grupo de control con el flujo tradicional; la evidencia diferencial es el argumento más sólido para continuar o pivotar la inversión.

La integración de IA en sistemas existentes no es un proyecto de reescritura sino una extensión incremental con validación continua de valor. El próximo capítulo aborda el testing

de estos sistemas —cómo verificar que funcionan correctamente cuando la salida del LLM es no determinista y cuando los cambios del proveedor son imprevisibles.

"Make it work, make it right, make it fast." — Kent Beck. En AI Engineering aplicado a sistemas existentes, el orden es: hacer que la capacidad de IA funcione correctamente en un subconjunto del tráfico, luego integrarla limpiamente en la arquitectura con el adapter pattern y la separación de concerns, y finalmente optimizar para escala y costo cuando el valor está validado con datos reales.